

MBSD Assignment #4 A.Y. 2024/25

Purposes

- Integrate the one-pedal controller into a simulated Arduino Uno¹ microcontroller (μ C), resorting to SimulIDE².
- Interact with the μ C through its digital and analog interfaces.

Instructions

For instructions on how to use SimulIDE and the Platform Support Packages, follow the instructions **provided by Prof. Violante in the lectures.**

The delivery shall contain:

- The controller Simulink model is used to generate the firmware binary file (plus all the accompanying files needed to make it possible to generate the code again, like .m files containing initializations)
- The firmware binary file to be loaded into the simulated Arduino in SimulIDE
- The SimulIDE project file.
- The PDF or Microsoft Word version of the report.

Is available an example based on a Tank level controller in the folder

The deliverable has to be provided as a . ZIP file up to June 8th at 23:59. It shall also contain a brief report explaining the integration process using the following template. It is sufficient that only one of the group members uploads it.

¹ Arduino Uno Board Anatomy <https://docs.arduino.cc/tutorials/uno-rev3/BoardAnatomy>, last visited on 14/05/2025.

² SimulIDE, <https://www.simulide.com/p/home.html>, last visited on 14/05/2025.

Model-Based Software Design, A.Y. 2024/25

Assignment 4 Report

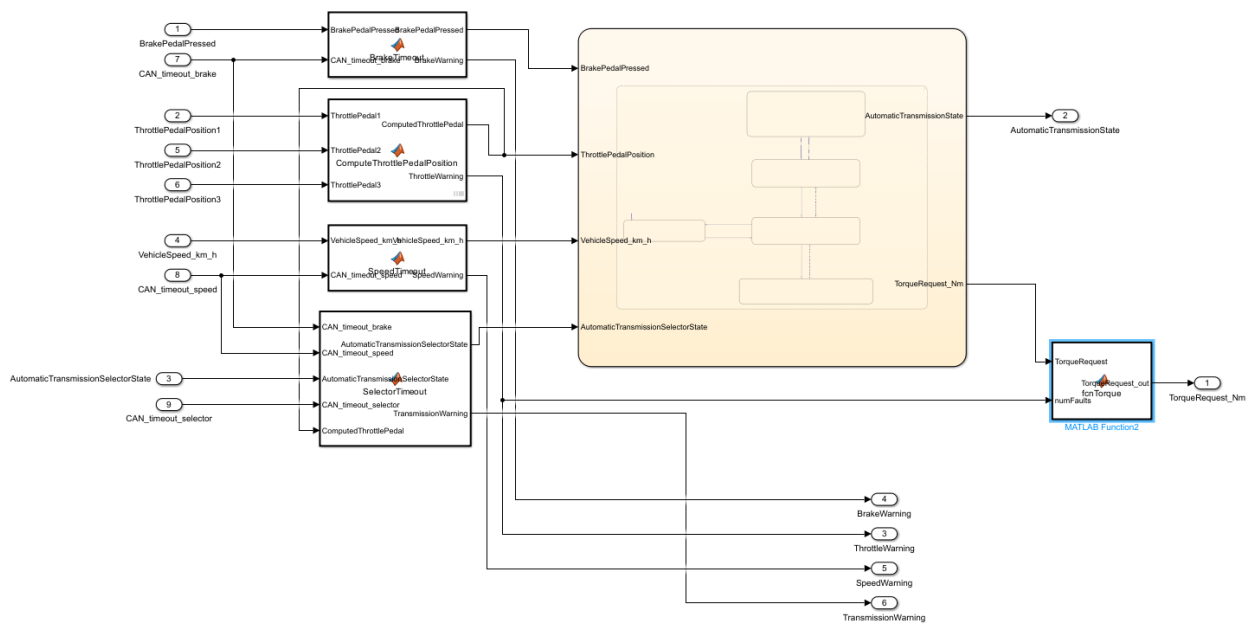
Components of the working group (max 2 people)

- Federica D'Ugo, 345772
- Francesca Fornasier, 338688

I/O interfaces

[Please describe the I/O interfaces to interact with the one pedal controller through the electrical interfaces of the Arduino]

[Screenshot of the controller model ready for the code generation]



Name	Unit	Type ³	Conversion formulas	Min ⁴	Max
ThrottlePedalPosition1	% (percentage of pedal pressed)	AI	$\text{ThrottlePedalPosition1} * (1/(2^{10}-1))$	0	1
ThrottlePedalPosition2	% (percentage of pedal pressed)	AI	$\text{ThrottlePedalPosition2} * (1/(2^{10}-1))$	0	1
ThrottlePedalPosition3	% (percentage of pedal pressed)	AI	$\text{ThrottlePedalPosition3} * (1/(2^{10}-1))$	0	1
BrakePedalPressed	0-1	DI		0	1
AutomaticTransmissionSelectorState	unitless	AI	$\text{AutomaticTransmissionSelectorState} * (1/(2^{10}-1))$	0	1
VehicleSpeed_kmh	km/h	AI	$\text{VehicleSpeed_km_h} * (300/(2^{10}-1)) - 60$	-60	240
CAN_timeout_brake	0-1	DI		0	1
CAN_timeout_speed	0-1	DI		0	1
CAN_timeout_selector	0-1	DI		0	1
TorqueRequest_Nm	Nm	DO (PWM)	$\text{TorqueRequest_Nm} * 255/160$	-80	80
AutomaticTransmissionState	unitless	DO		0	1
ThrottleWarning	0-1	DO		[]	[]
BrakeWarning	0-1	DO		[]	[]
SpeedWarning	0-1	DO		[]	[]
TransmissionWarning	0-1	DO		[]	[]

As part of the integration process, we reconsidered two MATLAB Function blocks involved in signal validation:

- `plausibility_check`, which originally generated warnings w1, w2, and w3 to flag throttle pedal inputs outside the valid range.
- `checkPlausibilityTorque`, which generated warning w4 to indicate if the calculated torque exceeded expected limits.

After analyzing the complete signal flow and safety logic, we decided to remove these warnings. In the `plausibility_check` function, any throttle pedal input found to be outside the valid range is assigned a value of -1. This value is then processed by the `computedThrottlePedalPosition` function, which performs a tolerance check across all three inputs. Since -1 lies well outside the chosen tolerance of ± 0.05 (given that throttle values are normally within the [0, 1] range), the affected input will be excluded from the average and a warning is triggered.

Furthermore, since the torque is computed based on signals that have already been validated, the resulting value is inherently constrained. As a result, performing an additional plausibility check at this stage, specifically through the `checkPlausibilityTorque` MATLAB

³ Digital Input (DI), Digital Output (DO), Analog Input (AI).

For AIs, provide the conversion formula from input voltage to the measurement unit data (indicating also how to perform the conversion from the raw reading of the ADC).

⁴ The Min/Max values that can be handled due to the conversion formula shall be expressed in the measurement unit specified in the Unit column.

function, would be redundant. For this reason, we decided to omit this function from the final implementation.

This simplification improves system clarity without compromising safety, and it will be detailed further in the following pages.

Accurate description of I/O interfaces :

- The three ThrottlePedalPositions were represented using Arduino analog input blocks, as their values range between 0 and 1. Since Arduino analog inputs return values between 0 and 1023, we applied a conversion by multiplying the input by $\frac{1}{2^{10}-1}$, in order to map it to the desired [0, 1] interval.
- The BrakePedalPressed signal was binary (0 or 1), so we implemented it using an Arduino digital input block.
- The AutomaticTransmissionSelectorState was represented as an analog input, since each driving mode corresponded to a numerical value in the "TransmissionState" Matlab function. Because the Arduino analog input block outputs values between 0 and 1023, we normalized this by multiplying it by $\frac{1}{2^{10}-1}$, resulting in a value between 0 and 1. This range was then divided into segments for the different driving modes using a dedicated Matlab function:

```
function transmissionState = double2enum(value)
    transmissionState = TransmissionState.Neutral;
    if(value>=0 && value<0.2)
        transmissionState = TransmissionState.Park;
    elseif(value>=0.2 && value<0.4)
        transmissionState = TransmissionState.Reverse;
    elseif(value>=0.4 && value<0.6)
        transmissionState = TransmissionState.Neutral;
    elseif(value>=0.6 && value<0.8)
        transmissionState = TransmissionState.Drive;
    elseif(value>=0.8 && value<=1)
        transmissionState = TransmissionState.Brake;
    end
```

- VehicleSpeed_kmh was represented as an Arduino analog input, as it could vary from -60 km/h to 240 km/h. To map the analog input to this 300 km/h range, we multiplied the value by $\frac{300}{2^{10}-1}$, then subtracted 60 to apply the correct offset and align it with the desired [-60, 240] interval.
- The three CAN_timeout signals were modeled using Arduino digital input blocks, as they were Boolean values used to indicate whether the corresponding data was outdated.
- TorqueRequest_Nm needed to vary between -80 Nm and 80 Nm. To convert this range into a signal suitable for an Arduino PWM output (since Arduino doesn't have analog output), we first added an offset of 80 to shift the range to [0, 160]. Then, we multiplied the result by $\frac{255}{160}$ to scale it to the [0, 255] range required by the PWM. This value was then used to control the duty cycle of a square wave, effectively representing the torque variation through the PWM output.
- AutomaticTransmissionState was transformed into a digital value using a Matlab function. It was set to true when the transmission state was Brake, and to false for all other modes. We chose to focus only on the Brake state, as it was the most significant for our application.

```
function atsooutput = funzione(ats)
    atsooutput=false;
    if (ats==TransmissionState.Brake)
        atsooutput=true;
    end
end
```

- ThrottleWarning, BrakeWarning, TransmissionWarning and SpeedWarning were implemented as Arduino digital output blocks, since they indicate whether an error is present (1) or not (0).
 - The ThrottleWarning was split into two digital outputs: one to signal the failure of a single channel and another to signal the failure of 2 channels. In our code, the throttle pedal is assigned 0 when no error is detected, 1 if one channel fails and 2 if both channels fail.
 - For the BrakeWarning, we assigned 0 for no error and 3 if an error is detected.
 - For the SpeedWarning, we used 0 when there is no error and 4 if an error occurs.
 - For the TransmissionWarning, we assigned 0 for no error and 5 if an error is detected.

Since each warning type uses a specific integer value to indicate an error, we created three separate Matlab functions to convert these values into binary outputs (0 or 1) for the digital Arduino blocks, as shown below:

```
function [err1, err2]= throttleLED(error)
    err1 = false;
    err2 = false;
    if(error>0)
        err1 = true;
    end
    if(error==2)
        err2 = true;
    end
end
```

```
function err6 = brakeLED(error6)
    err6 = false;
    if(error6>0)
        err6 = true;
    end
end
```

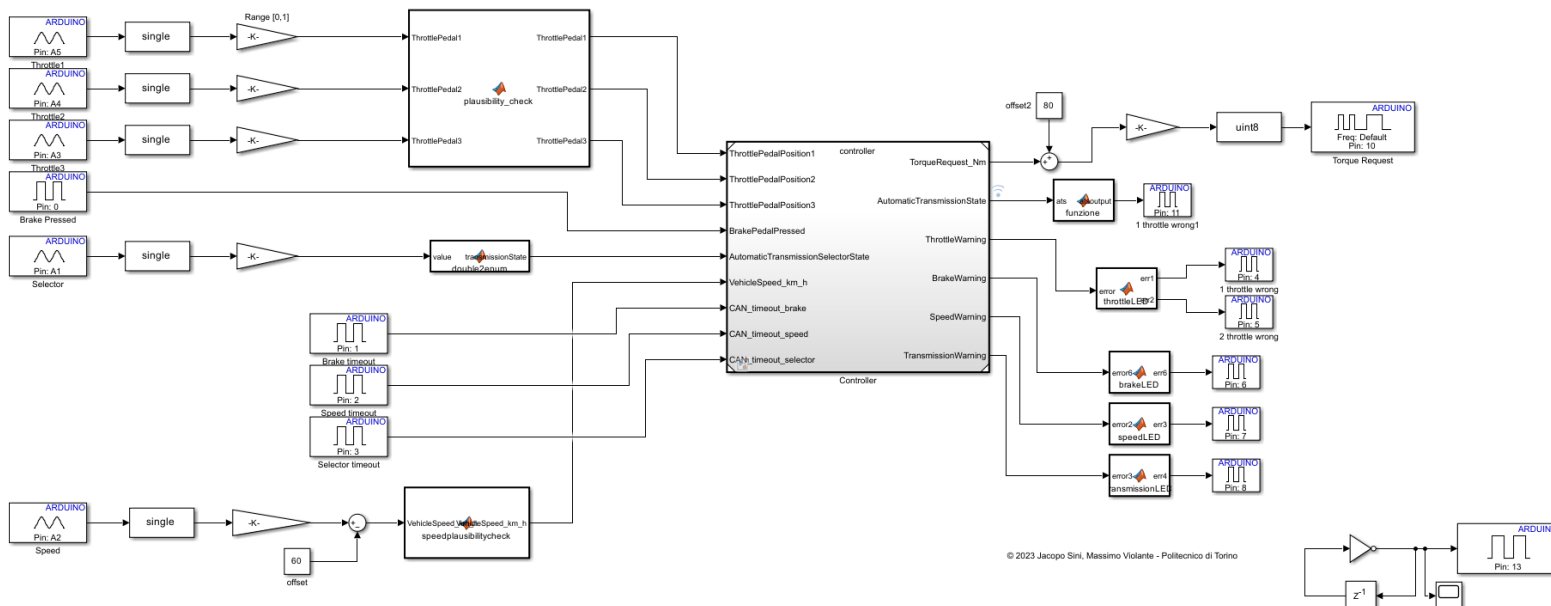
```
function err3 = speedLED(error2)
    err3 = false;
    if(error2>0)
        err3 = true;
    end
end
```

```
function err4 = transmissionLED(error3)
    err4 = false;
    if(error3>0)
        err4 = true;
    end
end
```

Code generation for Arduino

[Please describe the steps to generate the Arduino firmware using the Simulink Support Package for Arduino Hardware]

[screenshot of the controller model instrumented with the blocks of the Support Package for Arduino Hardware]



Steps to generate the Arduino firmware using the Simulink Support Package for Arduino Hardware:

First, we opened the Simulink controller model along with the model containing the Arduino inputs and outputs. We ensured that both models had the same settings, and configured them as follows:

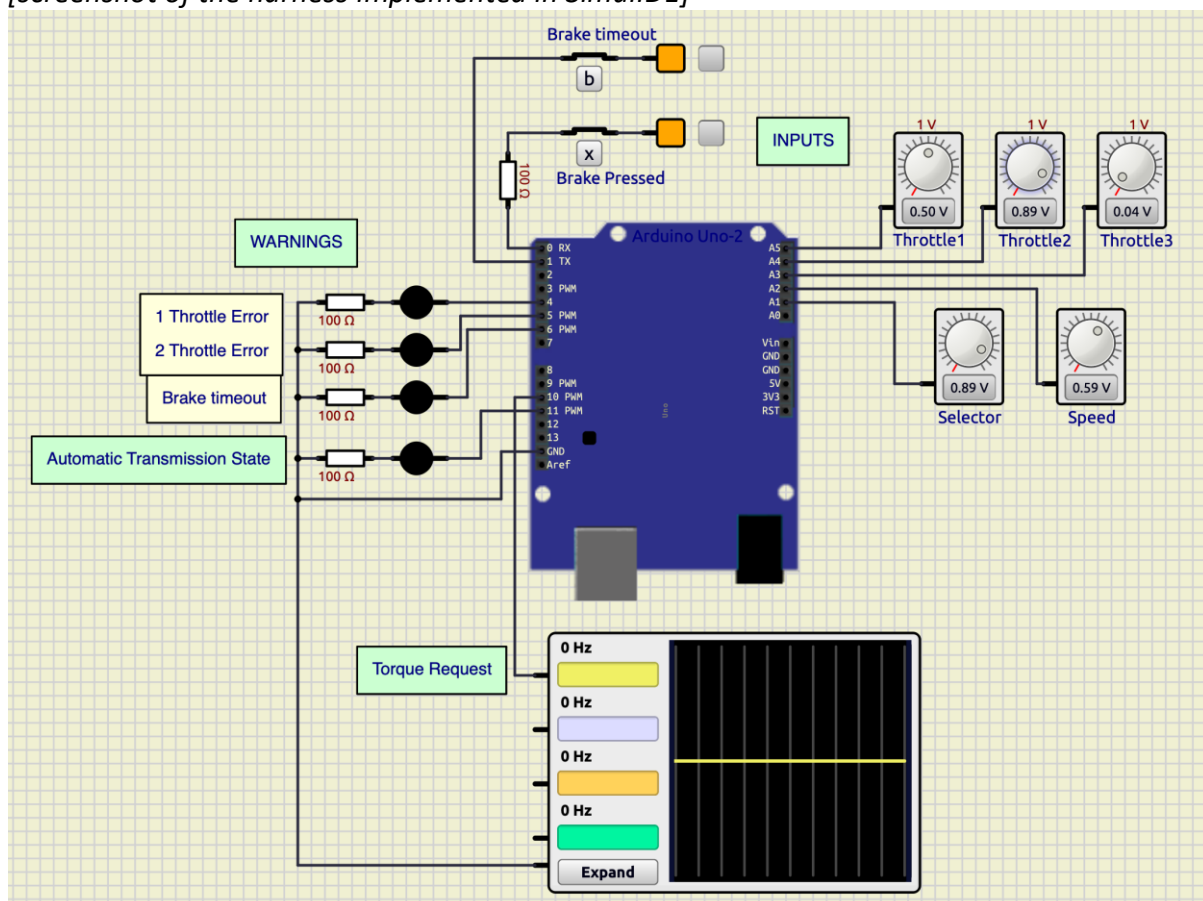
- Solver: *Fixed-step*, with type set to *Discrete (no continuous states)*
- Hardware Implementation: *Arduino Uno*
- Code Generation:
 - System target file: *ert.tlc*
 - Toolchain: *Arduino AVR*
 - Code placement: *Packaging format* set to *Compact*

Once the configuration was completed, we proceeded with the code generation by clicking on embedded encoder and build. This step compiled the model and generated the corresponding .hex file for each controller.

Then, in SimulIDE, we loaded the generated firmware: we right-clicked on the virtual Arduino board and selected the .hex file corresponding to our controller, in order to simulate the system with the actual compiled firmware.

Harness

[screenshot of the harness implemented in SimulIDE]



We chose not to include CAN_timeout_speed or CAN_timeout_selector in the SimulIDE project, keeping only CAN_timeout_brake to simplify the model as much as possible.

For the automatic transmission selector, we added a status LED that lights up only when the selected driving mode is Brake.

The brake pedal is modeled as a digital input: an open button corresponds to true, meaning the brake pedal is pressed. The same logic applies to CAN_timeout_brake, an open switch indicates that the CAN data are outdated, while a closed switch means the data are current. To simulate the analog inputs, we used five voltage sources: three for the throttle pedals, one for vehicle speed, and one for the transmission selector. Each input can be adjusted in real time by moving the on-screen potentiometer in SimulIDE.

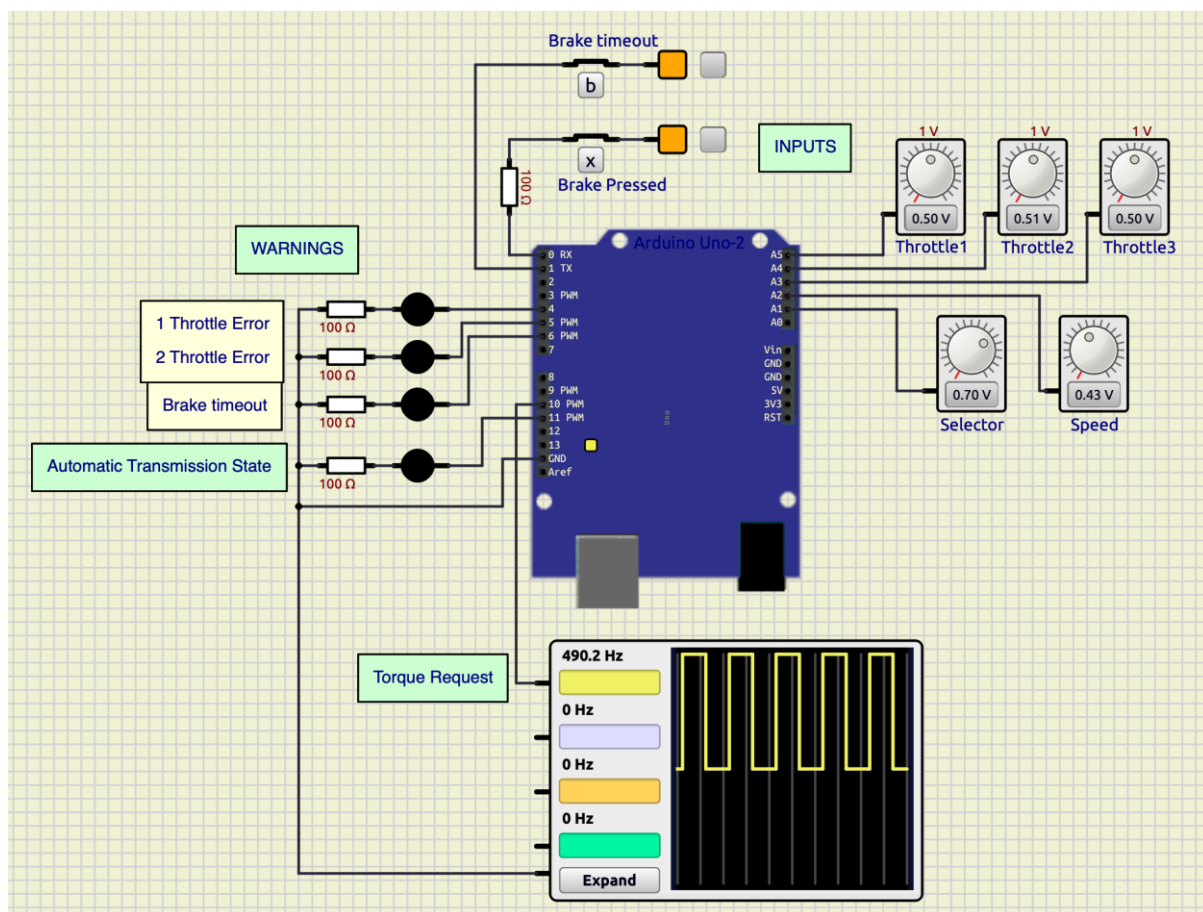
The Arduino controls three status LEDs:

- Fault LED 1 lights up when exactly one channel reports a fault.
- Fault LED 2 lights up when two channels report faults.
- Brake-timeout LED lights up when CAN_timeout_brake is open, indicating outdated brake data.

Since the Arduino doesn't have true analog outputs, the torque request is represented using a PWM signal. The torque range, from -80 Nm (regenerative braking) to $+80$ Nm, is linearly mapped to a duty cycle from 0% to 100%, which is then visualized on the oscilloscope

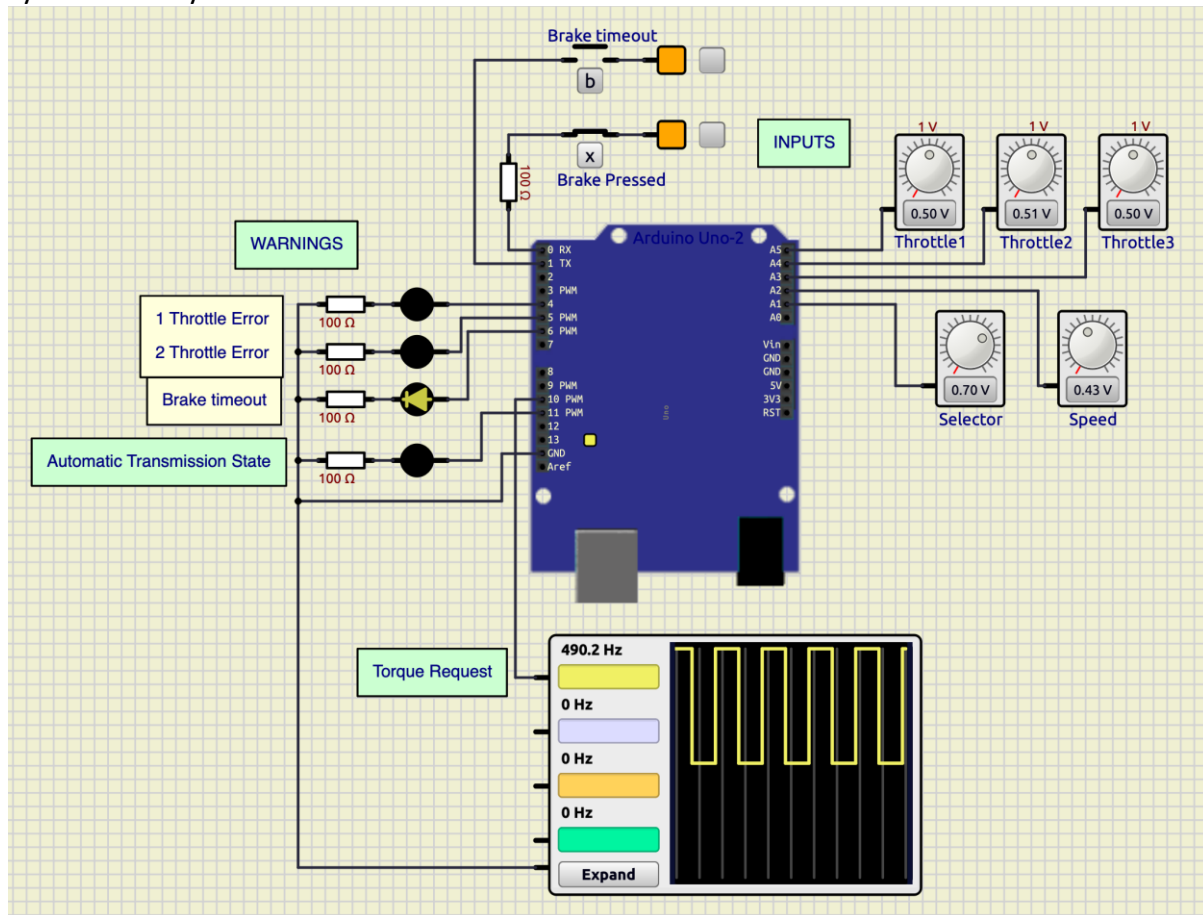
Test stimuli

[Provide a brief description of the stimuli set chosen to test the controller functionality] Due to the limitations of the SimulIDE, provide stimuli to check the function of the one pedal controller without the whole physical model

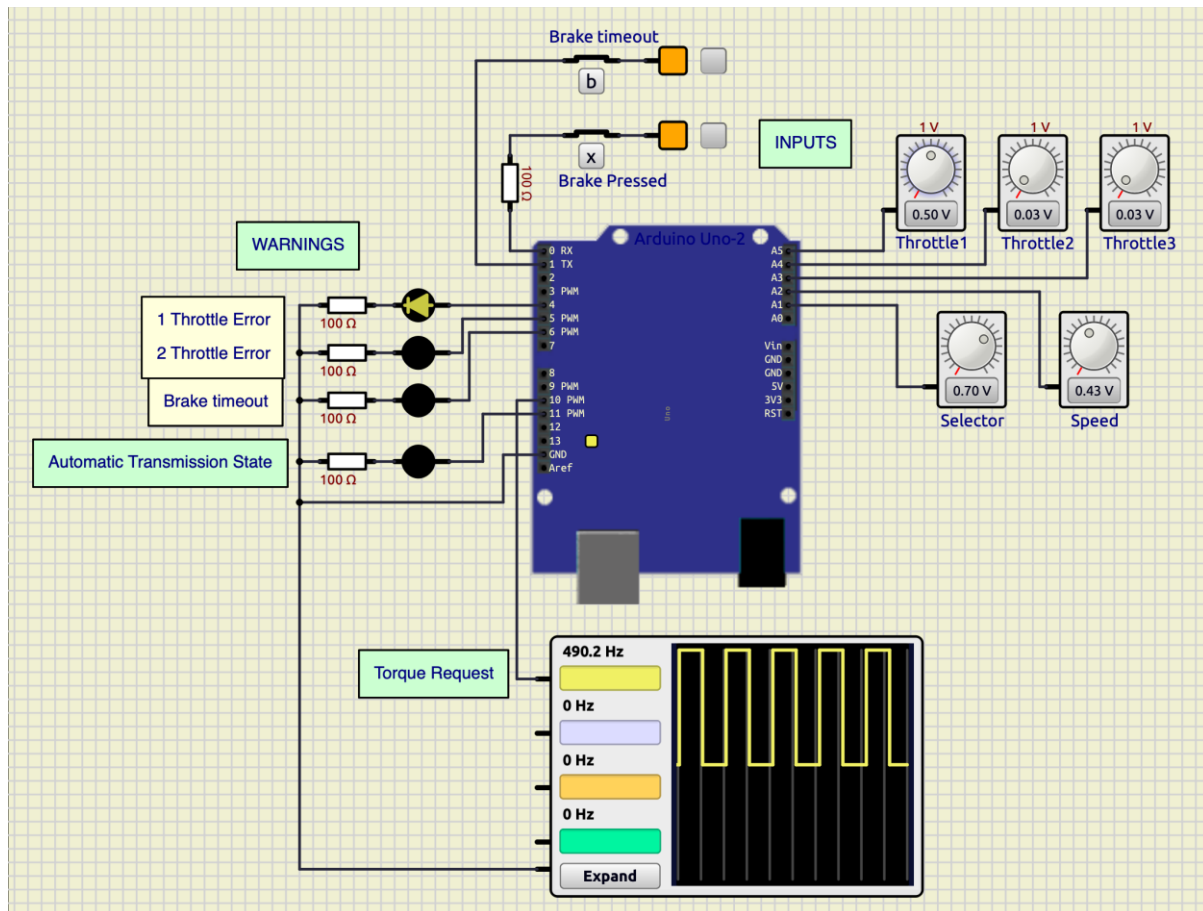


In the first test, we used three throttle values that are consistent (within a 5% tolerance), and both the brake pedal and the Brake Timeout signal are inactive, indicating that the brake is not engaged and the data is up to date. The Selector value is set to 0.7, corresponding to the Drive mode. As expected, no warnings are triggered (all output LEDs

remain off), and the duty cycle of the PWM signal representing the torque request is approximately 50%. This is consistent with a near-zero throttle position in Drive mode, resulting in a torque value close to zero (duty cycle of 50%). Additionally, the LED connected to pin 13 is implemented to blink continuously while the simulation is running, indicating system activity.



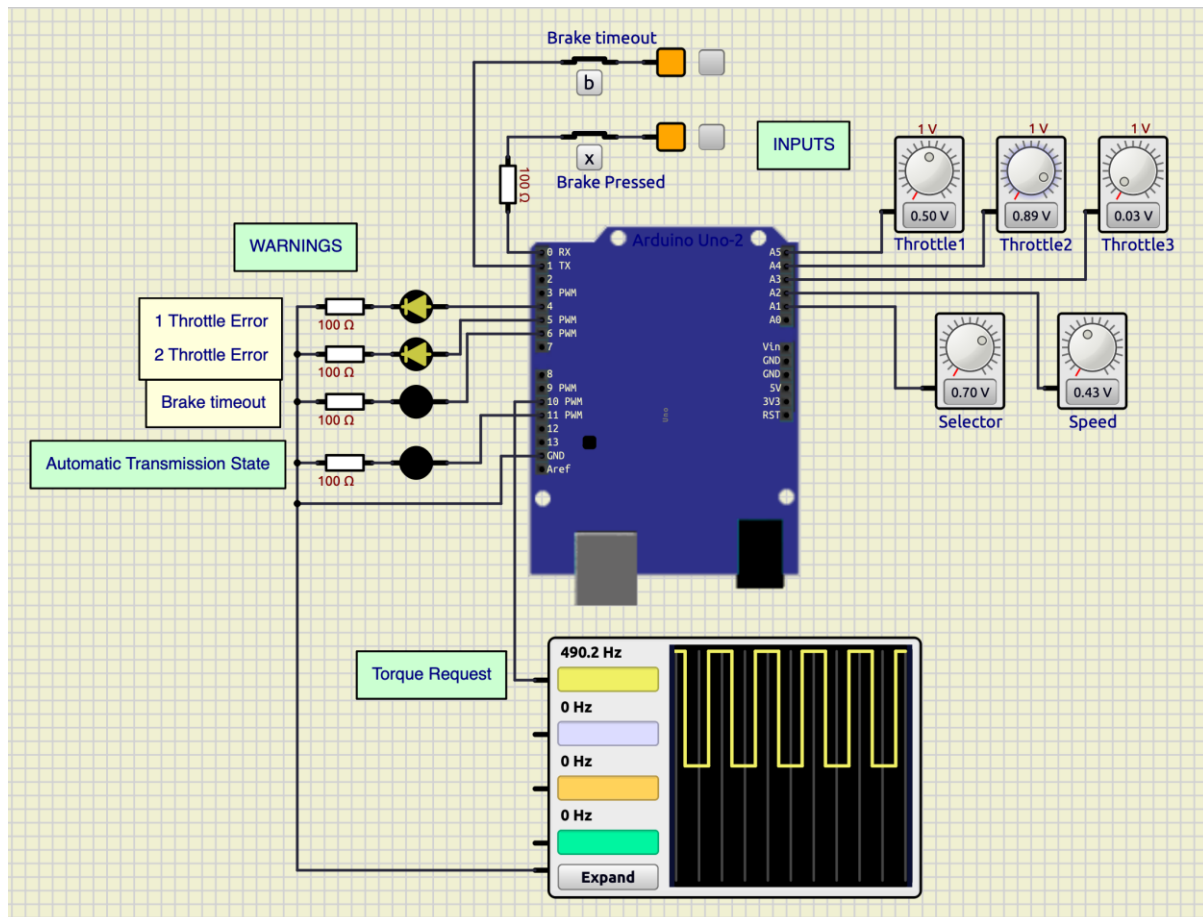
In this test, we verified the correct functionality of the Brake Timeout signal. The corresponding button was left open (setting the timeout variable to true), while all other input conditions were kept identical to the previous test case. As expected, the Brake Timeout LED turned on, indicating that the system responded correctly to the simulated timeout condition.



To verify the system's ability to detect inconsistencies among the throttle pedal signals, one of the three signals was set to a value of 0.5.

All other inputs were kept the same as in the previous test, except for the Brake Timeout signal: the corresponding button was pressed to indicate that the data is up to date, as in the first test case.

The system responded as expected: the LED indicating that one of the three throttle signals is inconsistent was correctly activated, confirming accurate detection of the discrepancy. Since only one of the throttle signals is incorrect, the system is still able to compute a valid torque request. As a result, no noticeable differences are observed in the torque output signal.



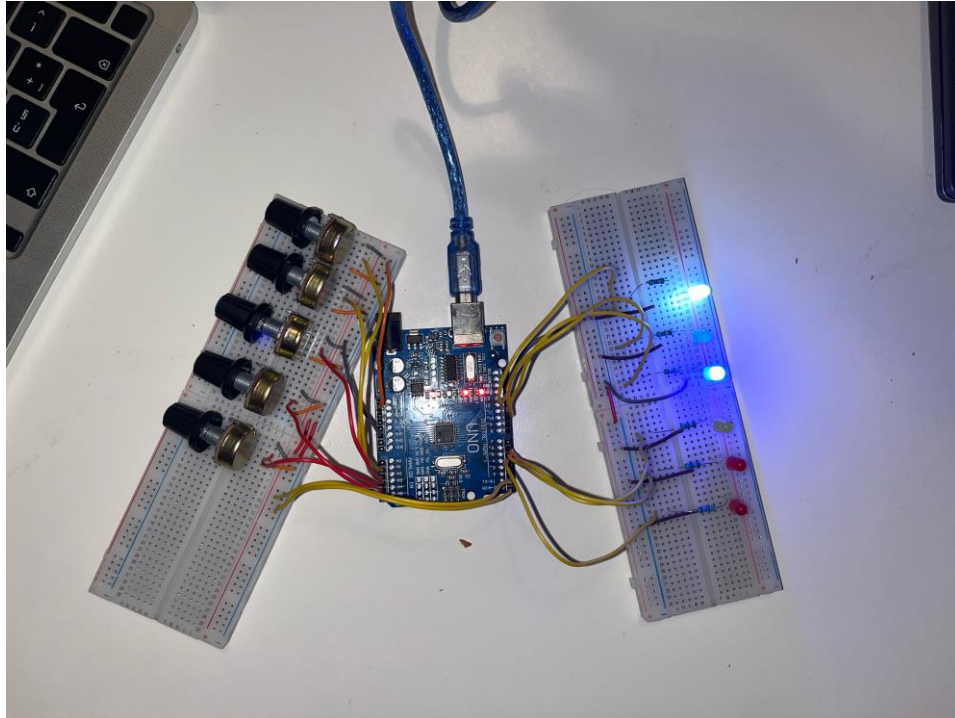
Finally, we tested the case in which at least two throttle signals are invalid, meaning all three signals differ from each other. To simulate this, Throttle3 was set to a value of 0.95, while all other inputs remained unchanged from the previous test. As expected, the second Throttle Error LED turned on, correctly indicating that two or more signals are inconsistent. In this condition, the torque request cannot be reliably computed and is therefore set to zero, in agreement with the previously defined ASIL safety requirements.

Implementation on a real ArduinoUno board

We uploaded the generated firmware to a physical Arduino Uno. Because no oscilloscope was available, we wired the Torque Request PWM output to an LED: the LED's blink rate and brightness follow the duty-cycle, giving a quick visual feedback of the commanded torque.

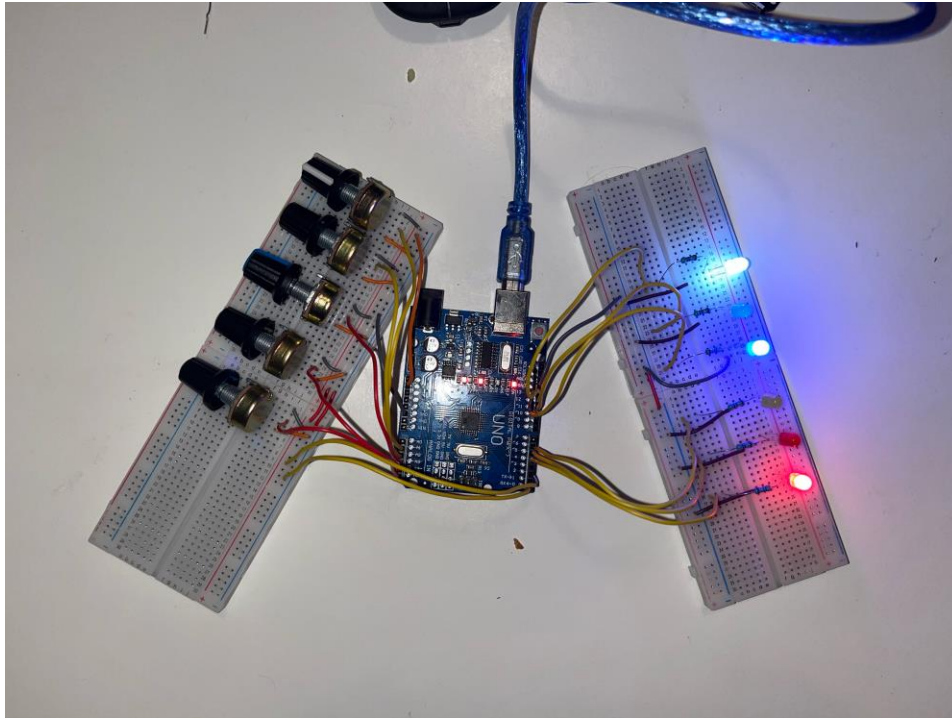
Test scenarios

1) All signals valid



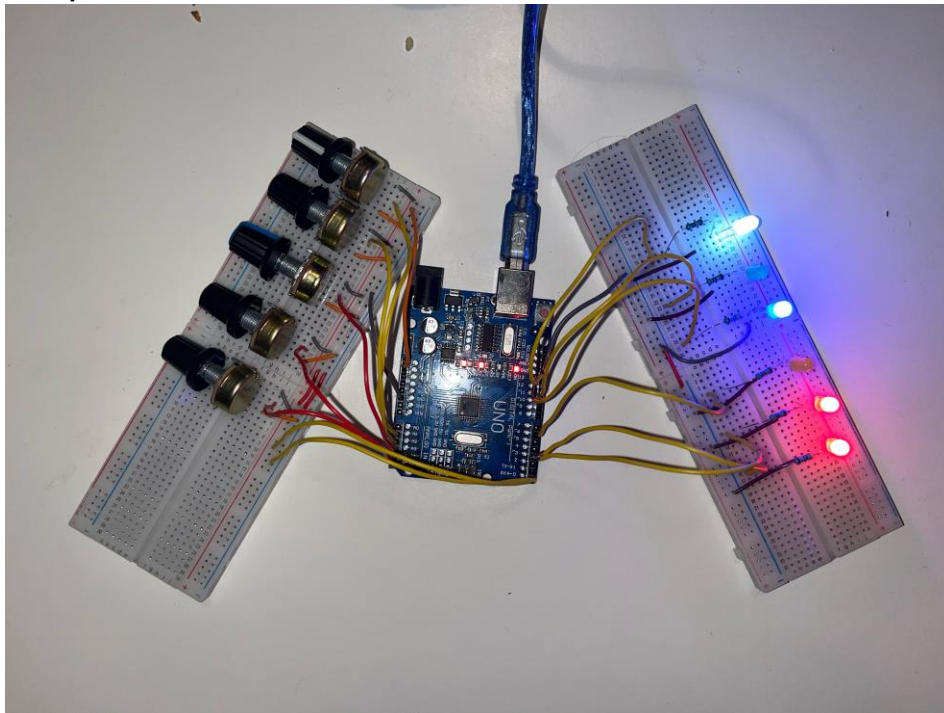
Since all the signals are valid, only the blinker LED and the PWM LED for torque are active, as expected.

2) Single-channel throttle fault



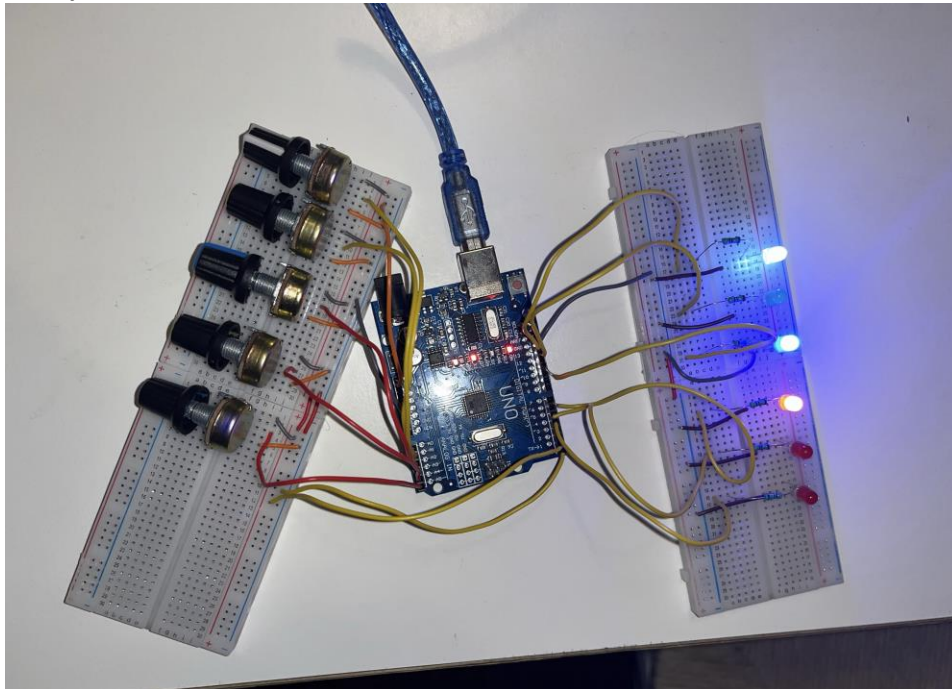
One throttle pedal is different from the other two, leading to the first throttle LED being switched on (red light).

3) Dual-channel throttle fault



The three throttle pedals are all different, causing also the second throttle led to be switched on (red led).

4) Brake-timeout condition



The digital input related to the brake timeout is connected to the voltage source, indicating the boolean value 'true'. The corresponding led (third one) is on.